Simple Circuit

Sensorless BLDC motor control with Arduino – DIY ESC

177 Comments / By Simple Projects / January 8, 2018

DIY ESC with Arduino UNO

This topic shows how to build a sensorless brushless DC (BLDC) motor controller or simply an ESC (Electronic Speed Controller) with an Arduino UNO board.
There are two types of brushless DC motors: sensored and sensorless. Sensored BLDC motor has built-in 3 hall effect sensors, these sensors detect the rotor position of the BLDC motor. Controlling a sensored BLDC motor is easy since we know the rotor position like what was done in the project below:
Sensored brushless DC motor control with Arduino

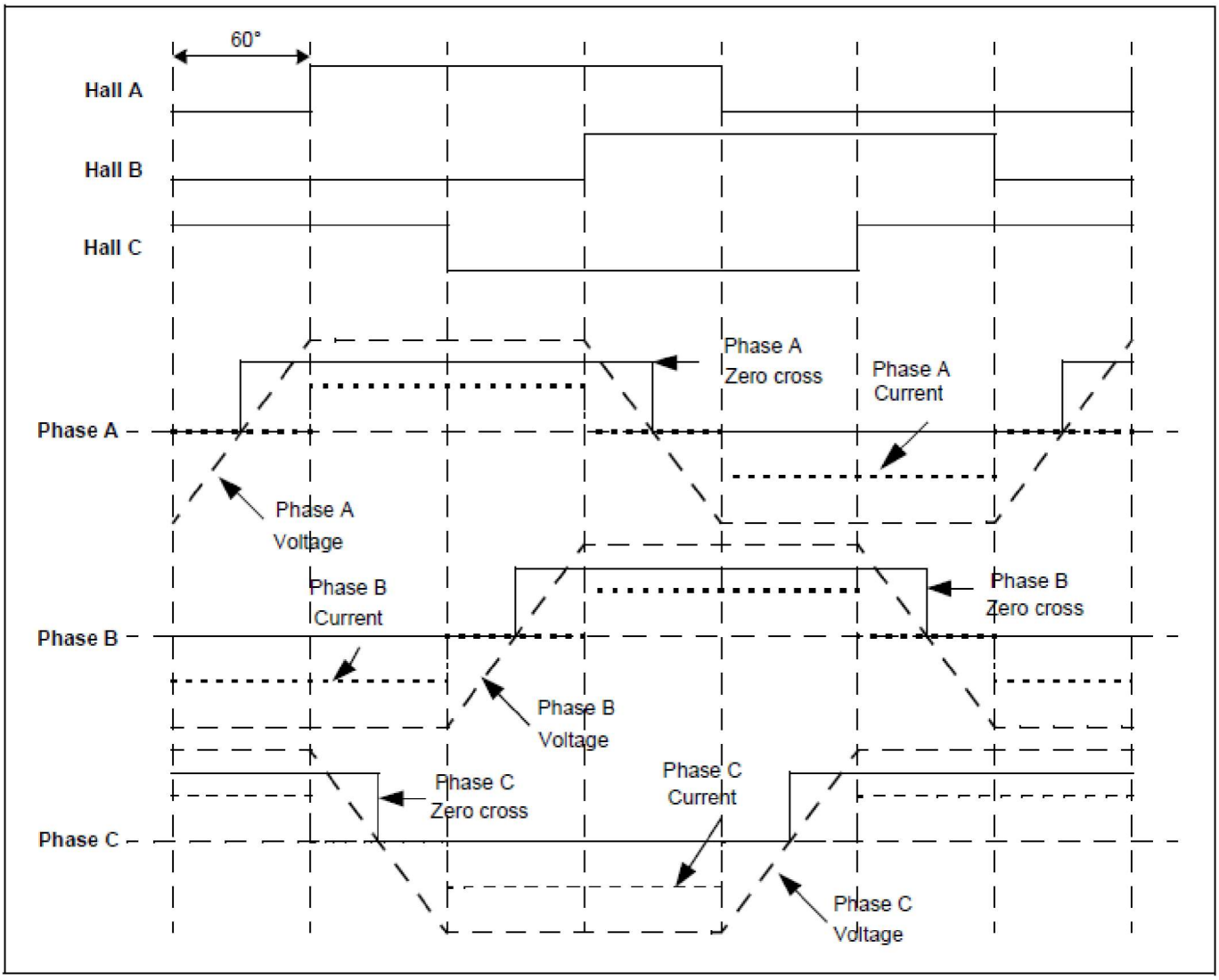
The commutation of the sensored BLDC motor is done according to the hall effect sensors state.

Sensorless BLDC motor doesn't have any sensor to detect its rotor position, its commutation is based on the BEMF (Back Electromotive Force) produced in the stator windings.

The main advantage of sensorless BLDC motor control is lower system cost and the main disadvantage is the motor must be moving at minimum rate to produce sufficient BEMF to be sensed.

How it works:

When the BLDC motor rotates, each winding (3 windings) generates BEMF opposes the main voltage. The 3 generated BEMF signals are 120° out of phase which is the same as the hall effect sensor signals. The figure below shows the relationship between the hall effect signals and the BEMF signals:



As shown in the figure above, the BEMF signals are not synchronized with the hall effect sensor signals (phase shift of 30°). In every energizing sequence, two windings are energized (one connected to positive and the other to negative) and the third winding is left open (floating). The floating winding is used to detect the zero crossing, thus, the combination of all 3 zero cross over point are used to generate the energizing sequence. Totally we've 6 events:

Phase A zero crossing: from high to low and from low to high

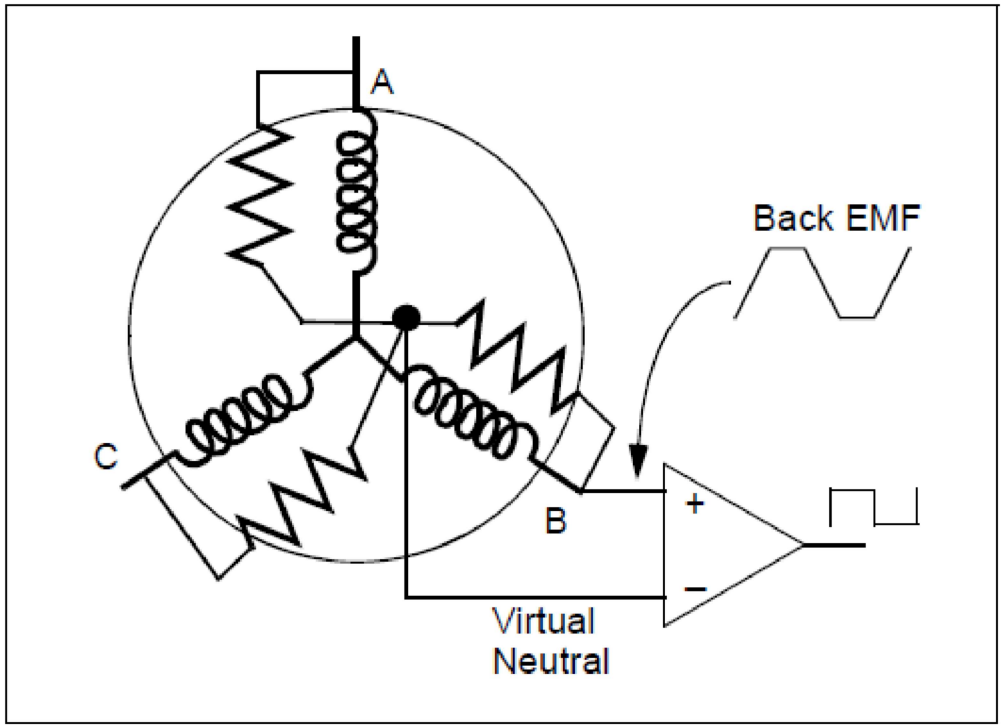
Phase B zero crossing: from high to low and from low to high

Phase C zero crossing: from high to low and from low to high

How to detect the zero crossing event:

The easiest way to detect the zero crossing events is by using comparators. The comparator has 3 main terminals: 2 inputs (positive and negative) and an output. Comparator output is logic high if the positive voltage is greater than the negative voltage, and logic low if the positive voltage is lower than the negative voltage.

Basically 3 comparators are needed for this project, connections are done as shown in the figure below (example for phase B). Each phase requires a similar circuit.



The virtual natural point is the same for all the 3 comparators, it is generated using 3 resistors. When the BEMF generated in the floating (open) winding crosses the zero point towards positive side, the comparator output makes a transition from low-to-high. When the BEMF generated in the floating winding crosses the zero point towards negative side, the comparator output makes a transition from high-to-low. By having three such comparator circuits, one on each of the phases gives three digital signals corresponding to the BEMF signal in the windings. The combination of these three signals is used to derive the commutation sequence.

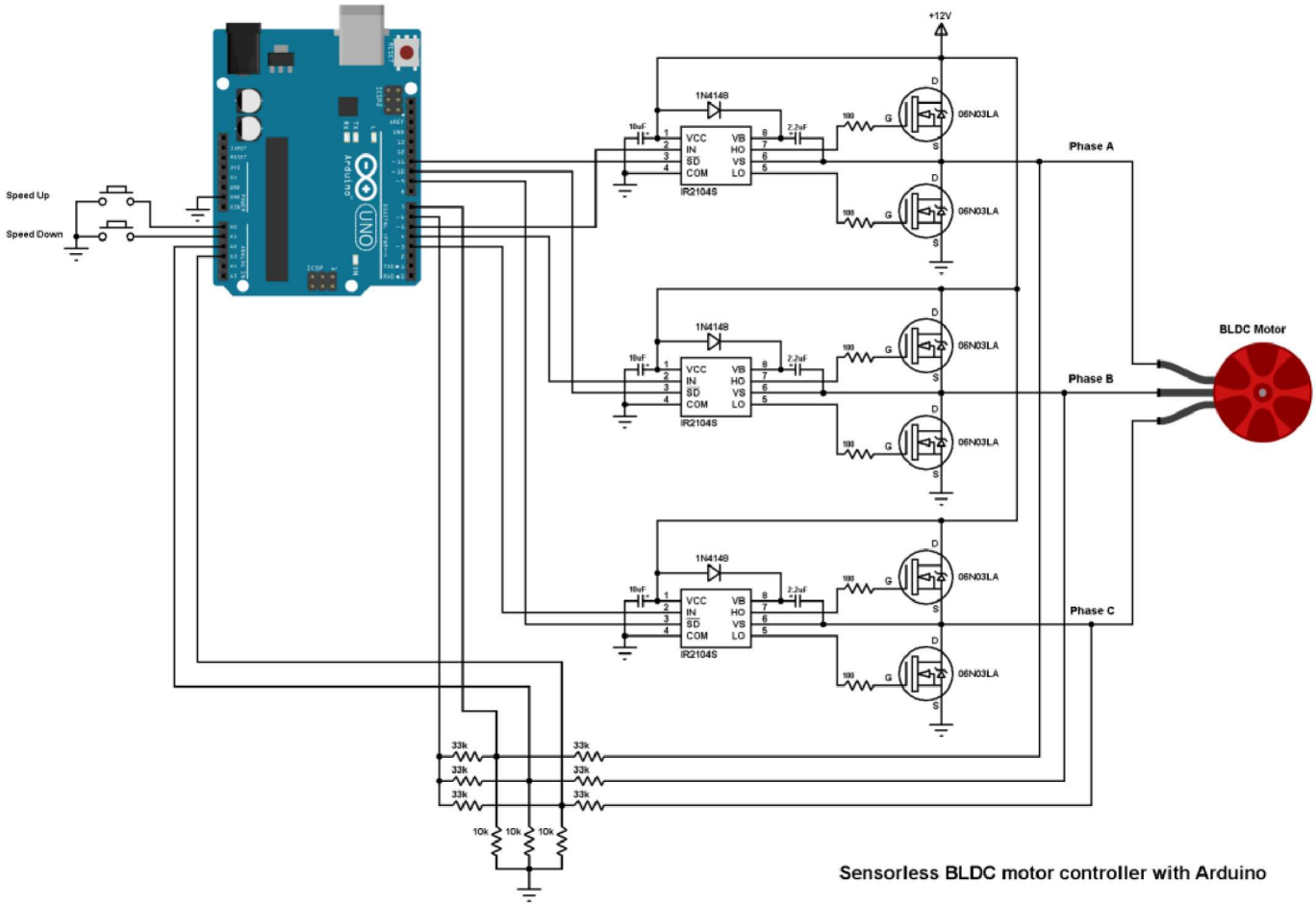
Hardware Required:

- Arduino UNO or similar board (Nano, Mini…) —> [ATmega328P datasheet]

- Brushless DC (BLDC) motor
- 6 x 06N03LA N-type mosfet (or equivalent) – datasheet
- 3 x IR2104S (IR2104) gate driver IC – datasheet
- 6 x 33k ohm resistor
- 3 x 10k ohm resistor
- 6 x 100 ohm resistor
- 3 x IN4148 diode
- 3 x 10uF capacitor
- 3 x 2.2uF capacitor
- 2 x pushbutton
- 12V source
- Breadboard
- Jumper wires

Sensorless BLDC motor control with Arduino circuit:

Project circuit schematic is shown below.



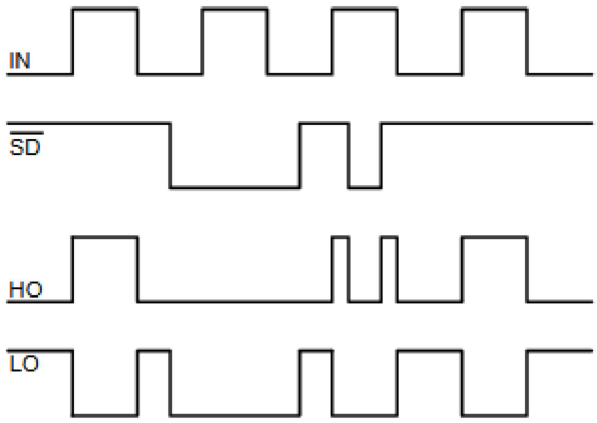
Note that all grounded terminals are connected together.

In the circuit there are 2 pushbuttons, one is used to increase BLDC motor speed and the 2nd one is used to decrease it.

The first three 33k (connected to motor phases) and the three 10k resistors are used as voltage dividers, because we can not supply the microcontroller with 12V, the other three 33k resistors generate the virtual natural point. The virtual natural point is connected to Arduino pin 6.

The Arduino UNO board is based on the ATmega328P microcontroller which has one analog comparator. The positive input of this comparator is on Arduino uno pin 6 (AIN0) and the negative input can be pin 7 (AIN1), A0 (ADC0), A1 (ADC1), A2 (ADC2), A3 (ADC3), A4 (ADC4) or A5 (ADC5). So I connected the virtual natural point to the positive pin of the analog comparator (pin 6), phase A BEMF to pin 7 (AIN1), phase B BEMF to pin A2 and phase C BEMF to pin A3. Each time the comparator compares the virtual point with the BEMF of one phase (this is done in the software). This minimizes the hardware needed and simplifies the circuit.

The IR2104S chips are used to control high side and low side mosfets of each phase. The switching between the high side and the low side is done according to the control lines IN and SD. The figure below shows input and output timing diagram:



Input/Output Timing Diagram

The SD lines of the three IR2104S are connected to pins 11, 10 and 9 respectively for phase A, phase B and phase C. The Arduino UNO can generate PWM signals on that pins where only high

side mosfets are PWMed.

Sensorless BLDC motor control with Arduino code:

The code below does not use any BLDC motor library.

As mentioned above, Arduino pins 9, 10 and 11 can generate PWM signals where pin 9 and pin 10 are related with Timer1 module (OC1A and OC1B) and pin 11 is related with Timer2 module (OC2A). Both Timer modules are configured to generate a PWM signal with a frequency of about 31KHz and a resolution of 8 bits. The duty cycles of the PWM signals are updated when a pushbutton is pressed (speed up or speed down) by writing to their registers (OCR1A, OCR1B and OCR2A).

The analog comparator compares the positive input AIN0 (Arduino pin 6) with the negative input which can be AIN1 (pin 7), ADC2 (pin A2) or ADC3 (pin A3). When the positive pin voltage is higher than the negative pin voltage, the output of the analog comparator ACO is set, and when the positive pin voltage is lower than the negative pin voltage, ACO is cleared.
In this project I used the analog comparator interrupt and I used its interrupt on rising (transition from low to high) and interrupt on falling (transition from high to low), this makes the zero crossing events interrupt the microcontroller.

To fully understand the code, please read the ATmega328 datasheet!

```
/* Sensorless brushless DC (BLDC) motor control with Arduino UNO (Arduino DI
 * This is a free software with NO WARRANTY.
 * https://simple-circuit.com/
 */

#define SPEED_UP          A0
#define SPEED_DOWN        A1
#define PWM_MAX_DUTY      255
#define PWM_MIN_DUTY      50
#define PWM_START_DUTY    100

byte bldc_step = 0, motor_speed;
unsigned int i;
void setup() {
  DDRD  |= 0x38;           // Configure pins 3, 4 and 5 as outputs
```

```
  PORTD  = 0x00;
  DDRB  |= 0x0E;           // Configure pins 9, 10 and 11 as outputs
  PORTB  = 0x31;
  // Timer1 module setting: set clock source to clkI/O / 1 (no prescaling)
  TCCR1A = 0;
  TCCR1B = 0x01;
  // Timer2 module setting: set clock source to clkI/O / 1 (no prescaling)
  TCCR2A = 0;
  TCCR2B = 0x01;
  // Analog comparator setting
  ACSR   = 0x10;           // Disable and clear (flag bit) analog comparator
  pinMode(SPEED_UP,   INPUT_PULLUP);
  pinMode(SPEED_DOWN, INPUT_PULLUP);
}
// Analog comparator ISR
ISR (ANALOG_COMP_vect) {
  // BEMF debounce
  for(i = 0; i < 10; i++) {
    if(bldc_step & 1){
      if(!(ACSR & 0x20)) i -= 1;
    }
    else {
      if((ACSR & 0x20))  i -= 1;
    }
  }
  bldc_move();
  bldc_step++;
  bldc_step %= 6;
}
void bldc_move(){        // BLDC motor commutation function
  switch(bldc_step){
    case 0:
      AH_BL();
      BEMF_C_RISING();
      break;
    case 1:
      AH_CL();
      BEMF_B_FALLING();
      break;
    case 2:
      BH_CL();
      BEMF_A_RISING();
      break;
    case 3:
```

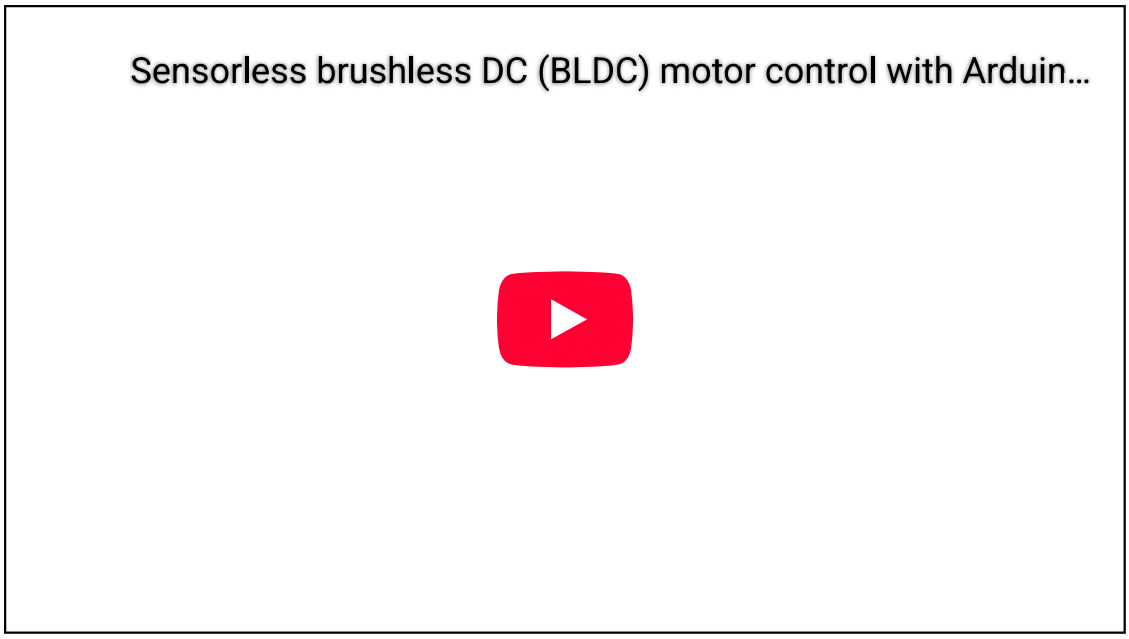
```
      BH_AL();
      BEMF_C_FALLING();
      break;
    case 4:
      CH_AL();
      BEMF_B_RISING();
      break;
    case 5:
      CH_BL();
      BEMF_A_FALLING();
      break;
  }
}

void loop() {
  SET_PWM_DUTY(PWM_START_DUTY);    // Setup starting PWM with duty cycle = P
  i = 5000;
  // Motor start
  while(i > 100) {
    delayMicroseconds(i);
    bldc_move();
    bldc_step++;
    bldc_step %= 6;
    i = i - 20;
  }
  motor_speed = PWM_START_DUTY;
  ACSR |= 0x08;                    // Enable analog comparator interrupt
  while(1) {
    while(!(digitalRead(SPEED_UP)) && motor_speed < PWM_MAX_DUTY){
      motor_speed++;
      SET_PWM_DUTY(motor_speed);
      delay(100);
    }
    while(!(digitalRead(SPEED_DOWN)) && motor_speed > PWM_MIN_DUTY){
      motor_speed--;
      SET_PWM_DUTY(motor_speed);
      delay(100);
    }
  }
}

void BEMF_A_RISING(){
  ADCSRB = (0 << ACME);    // Select AIN1 as comparator negative input
  ACSR |= 0x03;            // Set interrupt on rising edge
```

```
}
void BEMF_A_FALLING(){
  ADCSRB = (0 << ACME);    // Select AIN1 as comparator negative input
  ACSR &= ~0x01;           // Set interrupt on falling edge
}
void BEMF_B_RISING(){
  ADCSRA = (0 << ADEN);   // Disable the ADC module
  ADCSRB = (1 << ACME);
  ADMUX = 2;              // Select analog channel 2 as comparator negative
  ACSR |= 0x03;
}
void BEMF_B_FALLING(){
  ADCSRA = (0 << ADEN);   // Disable the ADC module
  ADCSRB = (1 << ACME);
  ADMUX = 2;              // Select analog channel 2 as comparator negative
  ACSR &= ~0x01;
}
void BEMF_C_RISING(){
  ADCSRA = (0 << ADEN);   // Disable the ADC module
  ADCSRB = (1 << ACME);
  ADMUX = 3;              // Select analog channel 3 as comparator negative
  ACSR |= 0x03;
}
void BEMF_C_FALLING(){
  ADCSRA = (0 << ADEN);   // Disable the ADC module
  ADCSRB = (1 << ACME);
  ADMUX = 3;              // Select analog channel 3 as comparator negative
  ACSR &= ~0x01;
}

void AH_BL(){
  PORTB  =  0x04;
  PORTD &= ~0x18;
  PORTD |=  0x20;
  TCCR1A =  0;            // Turn pin 11 (OC2A) PWM ON (pin 9 & pin 10 OFF)
  TCCR2A =  0x81;         //
}
void AH_CL(){
  PORTB  =  0x02;
  PORTD &= ~0x18;
  PORTD |=  0x20;
  TCCR1A =  0;            // Turn pin 11 (OC2A) PWM ON (pin 9 & pin 10 OFF)
  TCCR2A =  0x81;         //
}
```

```
void BH_CL(){
  PORTB  =  0x02;
  PORTD &= ~0x28;
  PORTD |=  0x10;
  TCCR2A =  0;            // Turn pin 10 (OC1B) PWM ON (pin 9 & pin 11 OFF)
  TCCR1A =  0x21;         //
}
void BH_AL(){
  PORTB  =  0x08;
  PORTD &= ~0x28;
  PORTD |=  0x10;
  TCCR2A =  0;            // Turn pin 10 (OC1B) PWM ON (pin 9 & pin 11 OFF)
  TCCR1A =  0x21;         //
}
void CH_AL(){
  PORTB  =  0x08;
  PORTD &= ~0x30;
  PORTD |=  0x08;
  TCCR2A =  0;            // Turn pin 9 (OC1A) PWM ON (pin 10 & pin 11 OFF)
  TCCR1A =  0x81;         //
}
void CH_BL(){
  PORTB  =  0x04;
  PORTD &= ~0x30;
  PORTD |=  0x08;
  TCCR2A =  0;            // Turn pin 9 (OC1A) PWM ON (pin 10 & pin 11 OFF)
  TCCR1A =  0x81;         //
}

void SET_PWM_DUTY(byte duty){
  if(duty < PWM_MIN_DUTY)
    duty  = PWM_MIN_DUTY;
  if(duty > PWM_MAX_DUTY)
    duty  = PWM_MAX_DUTY;
  OCR1A  = duty;                  // Set pin 9  PWM duty cycle
  OCR1B  = duty;                  // Set pin 10 PWM duty cycle
  OCR2A  = duty;                  // Set pin 11 PWM duty cycle
}
```

Sensorless BLDC motor control with Arduino video:

The video below shows how project is working.



References:

Microchip AN970 document

https://www.microchip.com/

Discover more from Simple Circuit

Subscribe to get the latest posts sent to your email.

Type your email... Subscribe

← Previous Post Next Post →

Recent Posts

Interfacing STM32 Blue Pill Board with ILI9341 TFT Display

Interfacing Arduino with SH1107 OLED Display in SPI Mode

Interfacing Arduino Board with SH1107 OLED Display in I2C Mode

Interface Nokia 5110 Graphic LCD with STM32 Blue Pill Board

STM32 Blue Pill with SH1107 OLED Display – I2C Mode Example

Sponsored Links:

PCBWay

Copyright © 2026 Simple Circuit

Home

Terms & Conditions

Privacy Policy

Contact